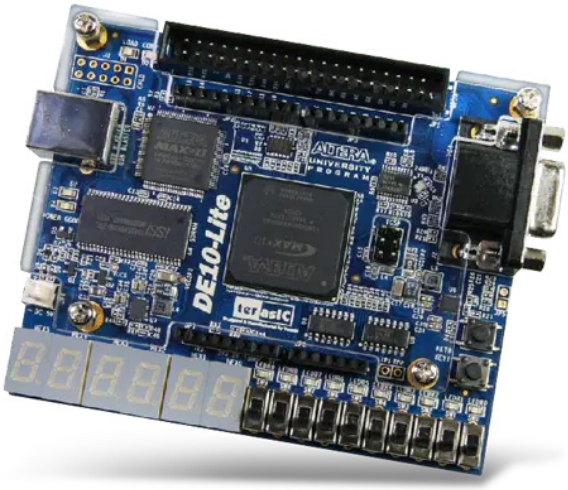
ISEL
Departamento de Engenharia
Eletrónica e Telecomunicações
e de Computadores

Licenciatura em Engenharia Informática e de Computadores e
Licenciatura em Engenharia Informática, Redes e Telecomunicações



RELATÓRIO DO 1º TRABALHO
Circuitos Combinatórios

Trabalho realizado por:

Gustavo Costa | Nº 52808

Rafael Pereira | Nº 52880

Turma: **LEIC15D**
Docente: **Prof.º Mário Véstias**

Lógica e Sistemas Digitais (LSD)
2024 / 2025 Semestre de Inverno

Conteúdo

Índice **i**

1 Objetivo **2**

2 Descrição do Circuito a Projetar **2**

3 Desenvolvimento do Projeto **3**

3.1 Funções lógicas dos circuitos 4

3.1.1 PZ4 4

3.1.2 PZ10 4

3.2 Simplificação lógica 5

3.3 Diagrama de blocos 6

3.4 Atribuição de Pinos 6

4 Resultados e Discussão **7**

4.1 Simulação 7

4.2 Teste 8

5 Conclusão **9**

Referências **10**

A VHDL **11**

A.1 PZ4.vhd 11

A.2 PZ10.vhd 12

B Esquema Elétrico **13**

C Índice Remissivo **14**

1 Objetivo

Este trabalho consiste em projetar, simular e implementar um circuito combinatório utilizando **VHDL** estrutural na placa **DE10-Lite da Intel**. O circuito irá verificar se o número de entradas ativas é **ímpar**, e, caso se verifique, irá **ativar a saída de paridade** (P). Se todas as entradas estão inativas, irá **ativar a saída zero** (Z). Para tal, será utilizada uma abordagem **hierárquica**, subdividindo o problema em blocos lógicos (3 blocos **PZ4** e 1 **AND** de 3 entradas) que verificam **paridade e zero** para subconjuntos das entradas, culminando na verificação de todas as 10 entradas da placa.

2 Descrição do Circuito a Projetar

O circuito a ser projetado é responsável por analisar as 10 entradas (inputs), que verifica se:

- O número de entradas ativas é **ímpar**, o que torna a saída da paridade **ativa (valor 1)**;
- O número de entradas ativas é **par**, o que torna a saída da paridade **inativa (valor 0)**;
- Todas as entradas estão **inativas**, o que torna saída de zero **ativa (valor 1)**;
- Pelo menos uma das entradas está **ativa**, o que torna a saída de zero **inativa (valor 0)**.

O circuito (PZ10) é constituído por **10 entradas** (I0, I1, I2, I3, I4, I5, I6, I7, I8 e I9), que correspondem a 10 botões da nossa placa de teste *DE10-Lite* da Intel e por **2 saídas** (P e Z), que serão interpretadas através de 2 LEDs da mesma placa.

Este circuito deve ser montado utilizando blocos funcionais que implementam um **verificador de paridade** e um **verificador de zero**, ambos com 4 entradas cada.

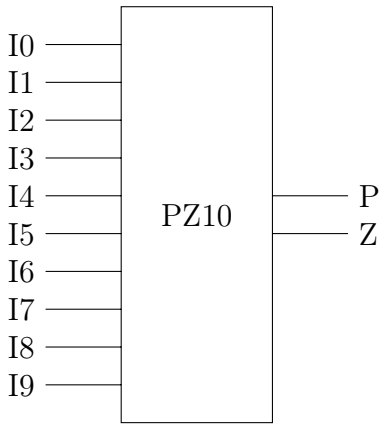


Figura 1: Diagrama disponibilizado no enunciado do circuito *PZ10*

3 Desenvolvimento do Projeto

Inicialmente, analisámos o circuito que nos foi pedido de acordo com o guião disponibilizado na plataforma *Moodle*, pelo professor Mário Véstias, e percebemos de que maneira iria se formular. A escolha do operador lógico **XNOR** baseou-se pelo facto de este operador apresentar apenas um output ativo, se e só se, os seus dois inputs forem iguais ((0 e 0) ou (1 e 1)).

Completando o raciocínio, uma vez que se pretendia que o output P ligasse **(valor 1)** apenas quando o número de entradas ativas fosse ímpar, concluímos que a utilização do operador lógico **XOR** para unir os dois **XNORs** usados neste mesmo circuito, seria o necessário para finalizar o pretendido. Ou seja, sempre que o número de entradas for ímpar, pela utilização de dois **XNORs**, o operador lógico **XOR** irá sempre receber um 1 e um 0, ou vice-versa, no que resultará num output final P ativo **(valor 1)**.

Caso contrário, se o número de entradas ativas for par, pela mesma utilização dos dois **XNORs**, o resultado será, ou dois outputs a 0 ou dois outputs a 1. Concluímos, então, que o operador lógico **XOR** irá devolver um output inativo **(valor 0)**.

Para o output Z, pretendia-se que este mesmo apenas ligasse quando todas as entradas estivessem inativas, ou seja, a 0. Para tal, optámos pela utilização do operador **NOR**, que consiste na negação do operador lógico **OR**. Este operador apenas resulta num output ativo quando ambas as entradas estão a 0. Por esta razão, para cada um destes outputs Z, conseguimos, através do operador lógico **NOR**, controlar que apenas passam 1s.

Finalizando, o operador lógico **AND** garantirá que o output final Z fique ativo, quando este operador receber apenas 1s. Isto é, o operador lógico **AND** só resulta num output ativo apenas se todas as entradas estiverem ativas **(valor 1)**. Logo a única combinação de inputs que resultará na ativação do output Z, será aquela em que todas as entradas estão desativadas **(valor 0)**, tal e qual como pedia o guião.

Tabela 1: Tabelas de Verdade dos operadores lógicos usados neste circuito

(a) XNOR

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

(b) XOR

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

(c) NOR

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

(d) AND

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

3.1 Funções lógicas dos circuitos

3.1.1 PZ4

$$P(I0, I1, I2, I3) = (\overline{I0 \oplus I1}) \oplus (\overline{I2 \oplus I3})$$

$$Z(I0, I1, I2, I3) = (\overline{I0 + I1}) \cdot (\overline{I2 + I3})$$

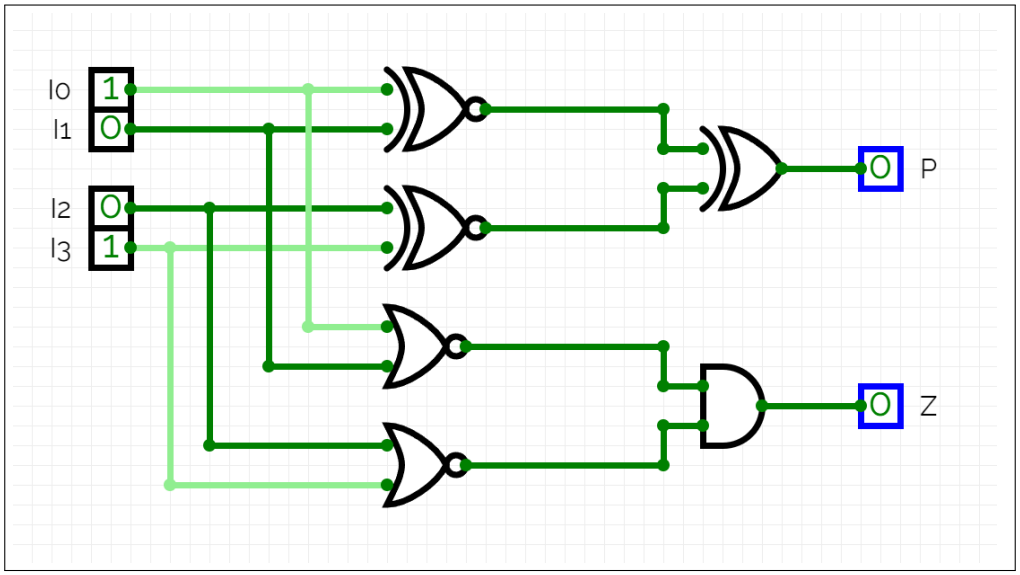


Figura 2: Projeção inicial do circuito *PZ4* criado no *Circuitverse*[1]

3.1.2 PZ10

$$P(I0, I1, \cdots, I8, I9) = (\overline{I0 \oplus I1}) \oplus (\overline{I2 \oplus I3}) \oplus (\overline{I4 \oplus I5}) \oplus (\overline{I6 \oplus I7}) \oplus (\overline{I8 \oplus I9})$$

$$Z(I0, I1, \cdots, I8, I9) = (\overline{I0 + I1}) \cdot (\overline{I2 + I3}) \cdot (\overline{I4 + I5}) \cdot (\overline{I6 + I7}) \cdot (\overline{I8 + I9})$$

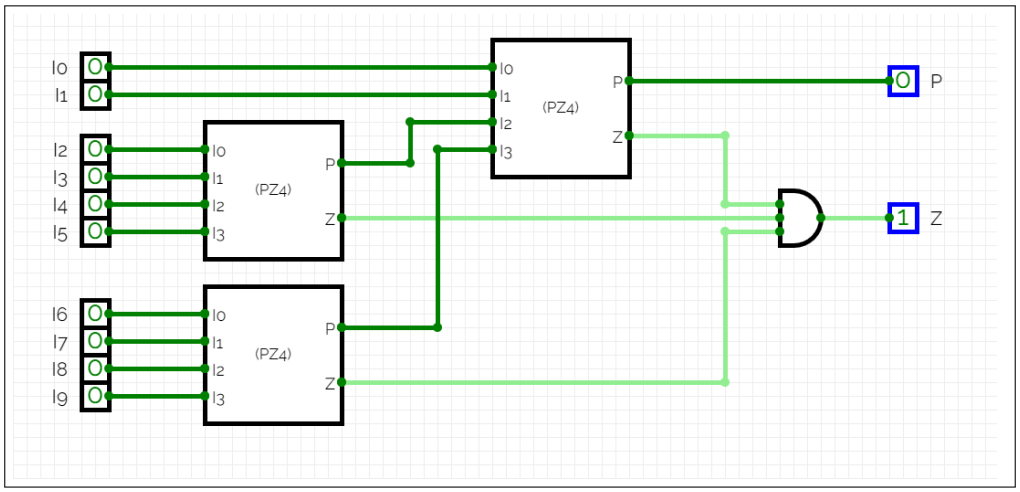


Figura 3: Projeção inicial do circuito *PZ10* criado no *Circuitverse*[2]

[1] ver referências[5] para site
[2] ver referências[5] para site

3.2 Simplificação lógica

Aplicando as propriedades dos operadores lógicos, conseguimos obter uma simplificação das funções lógicas dos circuitos.

Nas funções P, $PZ4_{U0}$, $PZ4_{U1}$ e $PZ4_{U2}$, exercemos uma propriedade do **XNOR**, que, uma vez que a função apresenta dois **XNORs**, isto é, um par de **XORs** negados, podemos remover a negação dos **XNORs** iniciais. E assim ficámos apenas com uma cadeia de **XORs**, que resultará no mesmo output.

No output **Z**, dos blocos *PZ4*, utilizámos simplesmente as Leis de De Morgan, que passaram de uma soma negada para uma multiplicação de negações, como se pode verificar na função a seguir.

A função **Z**, que funciona como um operador lógico **AND**, já se encontra simplificada, apenas recebendo como suas entradas, os saídas Z dos três blocos *PZ4* usados neste circuito. O output final **P** é, precisamente, a saída P do $PZ4_{U2}$.

PZ4

$$P(I0, I1, I2, I3) = (\overline{I0 \oplus I1}) \oplus (\overline{I2 \oplus I3}) = I0 \oplus I1 \oplus I2 \oplus I3$$
$$Z(I0, I1, I2, I3) = (\overline{I0 + I1}) \cdot (\overline{I2 + I3}) = \overline{I0} \cdot \overline{I1} \cdot \overline{I2} \cdot \overline{I3}$$

PZ10

$$PZ4_{U0}(I2, I3, I4, I5) = (\overline{I2 \oplus I3}) \oplus (\overline{I4 \oplus I5}) = I2 \oplus I3 \oplus I4 \oplus I5$$
$$PZ4_{U1}(I6, I7, I8, I9) = (\overline{I6 \oplus I7}) \oplus (\overline{I8 \oplus I9}) = I6 \oplus I7 \oplus I8 \oplus I9$$
$$PZ4_{U2}(I0, I1, P_{U0}, P_{U1}) = (\overline{I0 \oplus I1}) \oplus (\overline{P_{U0} \oplus P_{U1}}) = I0 \oplus I1 \oplus P_{U0} \oplus P_{U1}$$
$$P = P_{PZ4_{U2}}$$
$$Z(Z_{U0}, Z_{U1}, Z_{U2}) = Z_{U0} \cdot Z_{U1} \cdot Z_{U2}$$

3.3 Diagrama de blocos

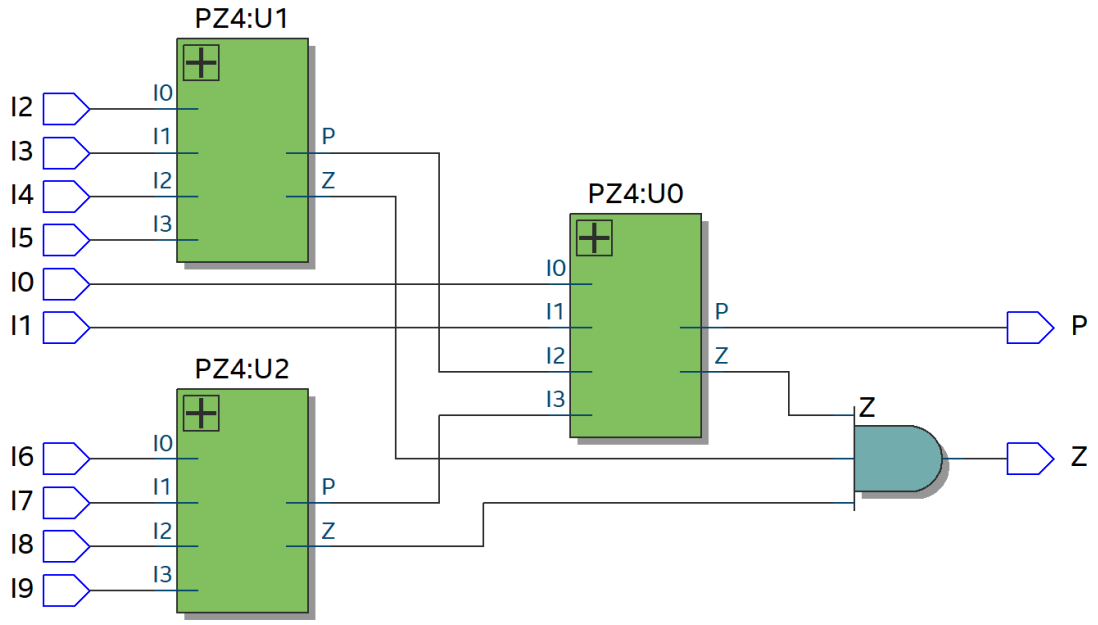


Figura 4: Diagrama do circuito *PZ10* criado pela funcionalidade ***RTL Viewer***

3.4 Atribuição de Pinos

A nossa atribuição aos pinos da placa de teste (***DE10-Lite***) de acordo com os 10 inputs (I0, I1, I2, I3, I4, I5, I6, I7, I8 e I9) e 2 outputs (P e Z) é a seguinte:

```
set_global_assignment -name BOARD "MAX 10 DE10 - Lite"
set_global_assignment -name DEVICE 10M50DAF484C6GES
set_global_assignment -name FAMILY "MAX 10"
set_location_assignment PIN_A8 -to P
set_location_assignment PIN_A9 -to Z
set_location_assignment PIN_C10 -to I0
set_location_assignment PIN_C11 -to I1
set_location_assignment PIN_D12 -to I2
set_location_assignment PIN_C12 -to I3
set_location_assignment PIN_A12 -to I4
set_location_assignment PIN_B12 -to I5
set_location_assignment PIN_A13 -to I6
set_location_assignment PIN_A14 -to I7
set_location_assignment PIN_B14 -to I8
set_location_assignment PIN_F15 -to I9
```

Figura 5: Código de atribuição de pinos

4 Resultados e Discussão

Logo após a conclusão definitiva do circuito em *VHDL* dentro do *Quartus*, simulámos, utilizando a ferramenta ***RTL Simulation***. Utilizando o *clock*, fomos subdividindo o valor para cada entrada, obtendo assim todas as combinações possíveis de entradas (I0..I9).

Verificámos então, que os valores dos outputs (P e Z) correspondentes a cada combinação de inputs, seguiam o proposto pelo guião deste trabalho e sempre apresentaram o valor esperado para quaisquer inputs. Assim, os resultados obtidos com as simulações do circuito foram os esperados, confirmando que o circuito cumpriu o objetivo inicial.

4.1 Simulação

I0	I1	I2	I3	I4	I5	I6	I7	I8	I9	P	Z
1	0	1	0	1	0	0	0	1	1	**1**	**0**
1	0	1	0	0	0	1	0	1	0	**0**	**0**
1	1	1	1	1	1	1	1	1	1	**0**	**0**
0	0	0	0	0	0	0	0	0	0	**0**	**1**
0	0	1	1	1	0	1	0	1	1	**0**	**0**
0	1	1	0	1	1	1	0	1	1	**1**	**0**
0	0	0	0	0	0	0	0	1	0	**1**	**0**
0	1	0	1	0	1	0	1	0	1	**1**	**0**
0	0	1	0	0	1	0	0	1	1	**0**	**0**
1	1	0	1	0	1	1	1	0	1	**1**	**0**

Tabela 2: Combinações[3] da *Testbench*

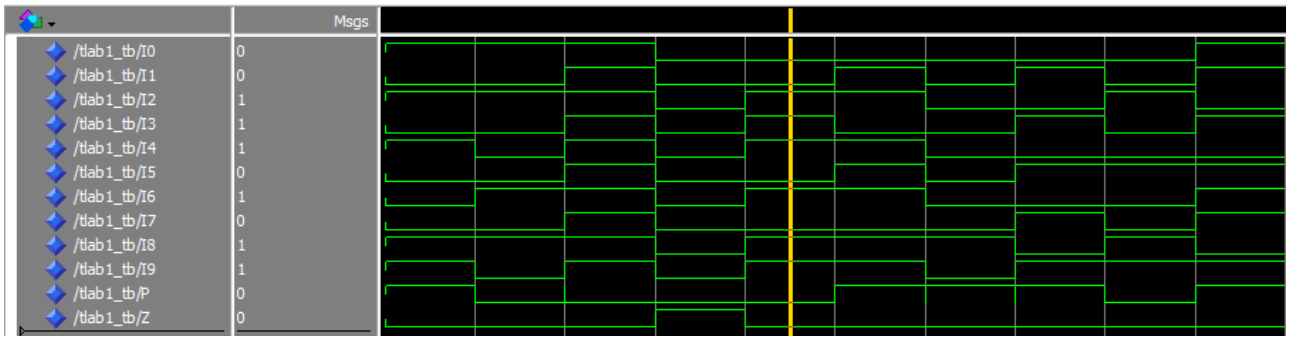


Figura 6: Visualização dos resultados da *Testbench*

[3]O caso indicado a amarelo faz juz à combinação demonstrada pela agulha a amarelo na Figura 6.

4.2 Teste

Apresentamos dois testes distintos, realizados em aula, com a placa *DE10-Lite* :

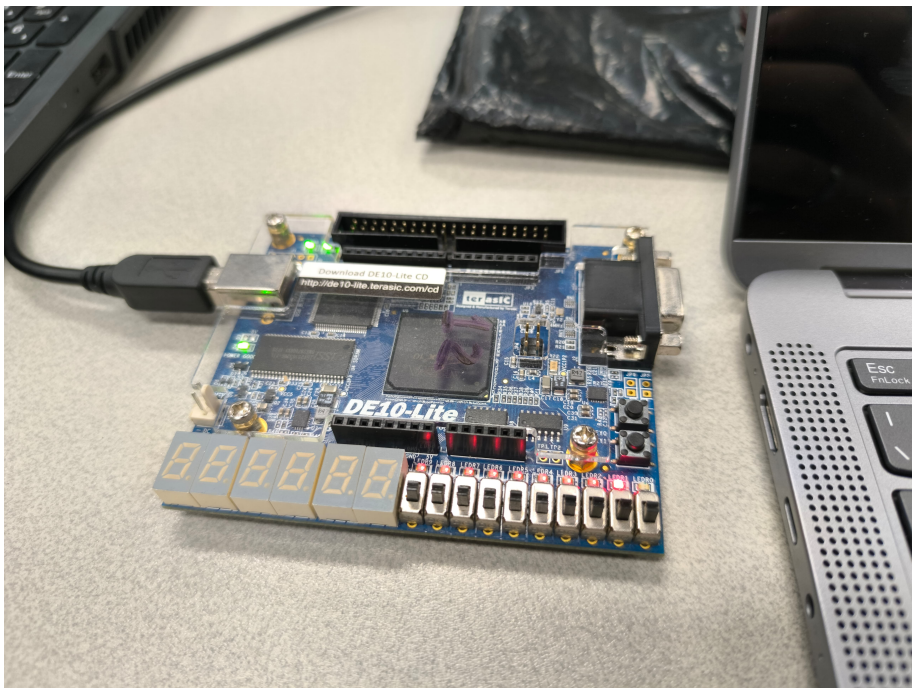


Figura 7: Teste para Zero[4] **(Z)**

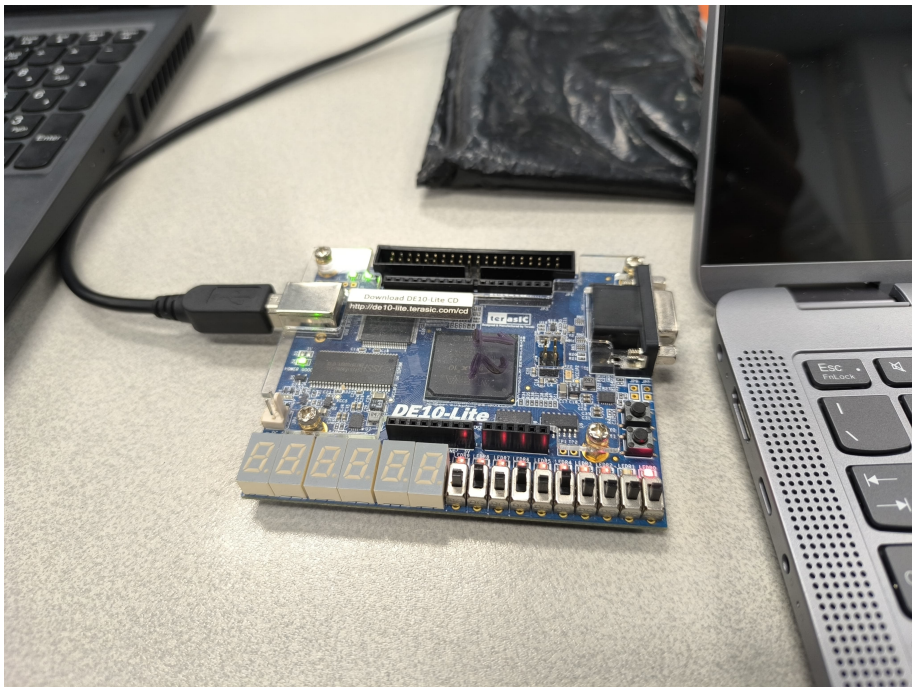


Figura 8: Teste para Paridade[5] **(P)**

[4]Teste com todos os inputs inativos. O último LED corresponde ao output **P** que se encontra inativo **(valor 0)**. O penúltimo LED corresponde ao output **Z** que se encontra ativo **(valor 1)**

[5]Teste com os inputs I5, I7 e I9 ativos. O último LED corresponde ao output **P** que se encontra ativo **(valor 1)**. O penúltimo LED corresponde ao output **Z** que se encontra inativo **(valor 0)**

5 Conclusão

De acordo com os objetivos propostos, concluímos com sucesso o desenvolvimento do circuito destinado à verificação de paridade e estado nulo das entradas. Através da utilização da linguagem ***VHDL*** no software ***Quartus*** da ***Intel***, aliada à implementação prática na placa ***DE10-Lite***, foi possível desenvolver um **circuito** que responde de forma **eficaz** e **precisa** aos critérios estabelecidos.

O circuito verifica a paridade das entradas, ativando o output **P** quando o número de entradas ativas **(valor 1)** é **ímpar**, e, adicionalmente, garante a ativação do output **Z** sempre que **não há** entradas ativas.

Ao longo do processo, procedemos a uma fase intensiva de testes na placa *DE10-Lite*, os quais confirmaram a **exatidão** do funcionamento do circuito. Os resultados experimentais foram consistentes e corroboraram plenamente as expectativas iniciais, **validando** a implementação do circuito e **confirmando** a correção dos outputs **P** e **Z** nas diferentes condições de entrada testadas.

Além de termos cumprido as especificações fornecidas no guião, esta experiência permitiu-nos **aprofundar** conhecimentos sobre a **aplicação prática** de circuitos digitais em contextos de **verificação** de paridade e **detecção** de entradas nulas. O projeto também nos proporcionou um maior domínio das ferramentas de desenvolvimento ***VHDL*** e da plataforma ***Quartus***, ao mesmo tempo que reforçou a nossa capacidade de **interpretar** e **implementar** soluções de acordo com as especificações exigidas.

Em suma, o trabalho realizado não só **cumpriu** os requisitos teóricos e práticos propostos, como também ampliou a nossa compreensão sobre a relevância e a eficácia dos circuitos digitais em problemas reais de verificação lógica.

Referências

- **Circuitverse**: Usado para a simulação inicial e esquematização do circuito;
- **Overleaf Documentation**: Usada em recurso para desenvolvimento do documento em *LaTeX*;
- **Rice University *LaTeX* Mathematical Symbols**: Documento da Universidade Rice, no Texas, dedicado demonstrar símbolos e técnicas de formatação em *LaTeX* menos usados e que nos muito útil ao desenvolvimento do documento;
- **UC Irvine Department of Mathematics *LaTeX* Math Symbols**: Documento da Universidade da Califórnia, na cidade de Irvine, que contém uma compilação de símbolos, gráficos, tabelas e entre outros muito úteis para o desenvolvimento deste documento em *LaTeX*;
- ***VHDL* Estrutural e Simulação - Professor Mário Véstias**: Material disponibilizado pelo Professor Mário Véstias, do ISEL. Este documento serve como referência para o desenvolvimento de modelos estruturais e a realização de simulações em *VHDL*;
- **Simplificação Lógica: Método de Karnaugh - Professor Mário Véstias**: Documento fornecido pelo Professor Mário Véstias, do ISEL, utilizado para explicar o processo de simplificação lógica através dos Mapas de Karnaugh.

A VHDL

A.1 PZ4.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity PZ4 is
    port (
        I0, I1, I2, I3: in std_logic;
        P, Z: out std_logic);
end PZ4;

architecture arc_pz4 of PZ4 is

begin

    P <= (I0 xnor I1) xor (I2 xnor I3);
    Z <= (I0 nor I1) and (I2 nor I3);

end arc_pz4;
```

A.2 PZ10.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity PZ10 is
    port (
        I0, I1, I2, I3, I4, I5, I6, I7, I8, I9: in std_logic;
        P, Z: out std_logic);
end PZ10;

architecture arc_PZ10 of PZ10 is
component PZ4
    port (
        I0, I1, I2, I3: in std_logic;
        P, Z: out std_logic);
end component;

    signal P1_temp, Z1_temp: std_logic;
    signal P2_temp, Z2_temp: std_logic;
    signal P_temp, Z3_temp: std_logic;

begin

    U0: PZ4
        port map (I0 => I2, I1 => I3, I2 => I4,
        I3 => I5, P => P1_temp, Z => Z1_temp);

    U1: PZ4
        port map (I0 => I6, I1 => I7, I2 => I8,
        I3 => I9, P => P2_temp, Z => Z2_temp);

    U2: PZ4
        port map (I0 => I0, I1 => I1, I2 => P1_temp,
        I3 => P2_temp, P => P_temp, Z => Z3_temp);

    P <= P_temp;
    Z <= (Z1_temp and Z2_temp and Z3_temp);

end arc_PZ10;
```

B Esquema Elétrico

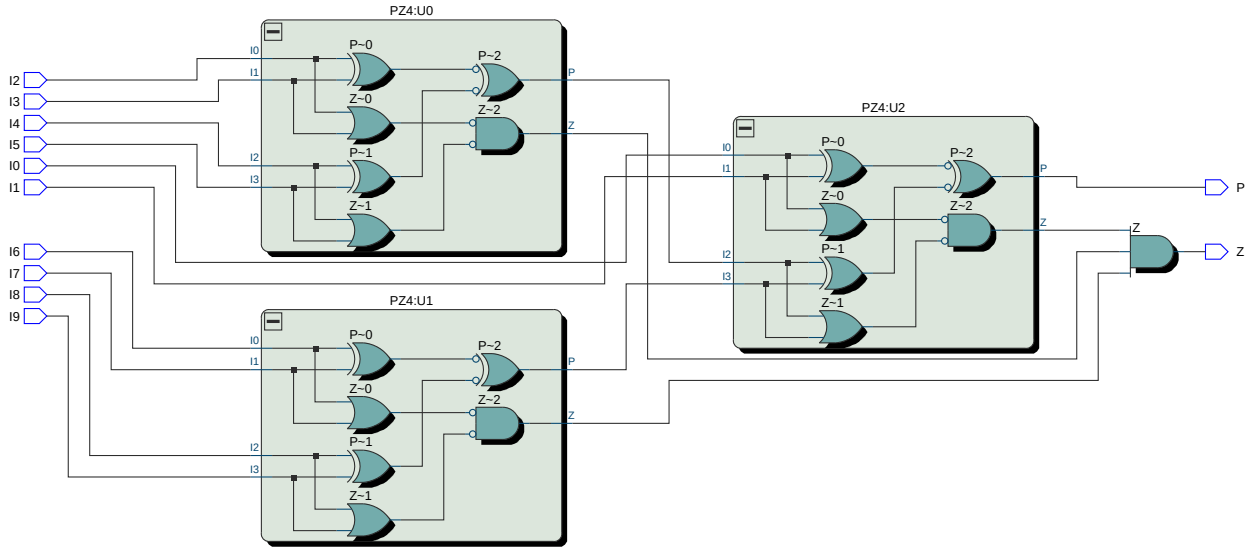


Figura 9: Esquema elétrico do circuito *PZ10*

C Índice Remissivo

Lista de Figuras

1 Diagrama disponibilizado no enunciado do circuito *PZ10* 2
2 Projeção inicial do circuito *PZ4* criado no *Circuitverse*[6] 4
3 Projeção inicial do circuito *PZ10* criado no *Circuitverse*[7] 4
4 Diagrama do circuito *PZ10* criado pela funcionalidade ***RTL Viewer*** 6
5 Código de atribuição de pinos 6
6 Visualização dos resultados da *Testbench* 7
7 Teste para Zero[8] **(Z)** 8
8 Teste para Paridade[9] **(P)** 8
9 Esquema elétrico do circuito *PZ10* 13

Lista de Tabelas

1 Tabelas de Verdade dos operadores lógicos usados neste circuito 3
2 Combinações[10] da *Testbench* 7